Homework 4

[2026-1] 데이터사이언스를 위한 컴퓨팅 (001)

Due: 2026 년 6 월 19 일 23:59

1. Coin Combinations [25pts]

Given a set of distinct positive integers representing coin values and a target `sum`, determine the number of ordered sequences of coins that sum to the target. A coin may be used multiple times. Implement the function `coinCombinations` in `functions.cpp` as specified below.

1.1. Inputs

- `int target`: A target sum
- `std::vector<int>& coins`: A vector containing a set of distinct coin values (c_1, c_2, . . . c_n), where each value is a positive integer.

1.2. Output

- The function returns the number of distinct ways to reach the sum. Return the count modulo $10^9 + 7$.

Constraints

```
1 ≤ target ≤ 10^6
1 ≤ coins.size() ≤ 100
```

Example

```
std::vector<int> coins = {1, 2, 3};
```

2. Flight Routes [25pts]

You are provided with a dataset representing flight routes between cities, and their costs. Your task is to implement the function `flightRoutes` to analyze the most cost-effective paths from San Francisco (City `1`) to New York (City `n`).

Implement the function in `functions.cpp` according to the following specifications:

1. Identify the **minimum cost** required to travel from City `1` to City `n`.
2. Identify **all** distinct paths that achieve this minimum cost.
3. From these specific minimum-cost paths, determine:
    a. The **total count** of such paths.
    b. The **minimum number of flights** (edges) involved.
    c. The **maximum number of flights** (edges) involved.

Print the minimum cost, the total count of paths, minimum and maximum number of flights.

2.1. Inputs

- `int n`: The number of cities.
- `std::vector<std::vector<int>>& routes`: A vector of routes representing the flight routes. Each element consists of the source city `u`, the destination city `v`, and the cost of the flight `c`. Each flight is one-way.

You can assume there exists at least one route from San Francisco to New York.

2.2. Output

The function should not return values. Instead, it should print four space-separated integers with `std::endl` at the end. Each value should be modulo $10^9 + 7$.

1. The minimum cost
2. The total count of distinct paths
3. The minimum number of flights among those paths
4. The maximum number of flights among those paths

Constraints

$1 \leq n \leq 10^5$
$1 \leq u, v \leq n$

Example

```
std::vector<std::vector<int>> routes = {{1,4,5}, {1,2,4},
{2,4,5}, {1,3,2}, {3,4,3}};
```

3. Height Ranks [50pts]

You are given a set of n students with unique heights and a list for height comparisons. A comparison `a < b` (represented as a directed edge a → b) indicates that student `a` is shorter than student `b`.

A student's rank is considered determinable if and only if they can be compared to every other student. That is, for a specific student S and every other peer P, it must be known whether S is taller than P or S is shorter than P.

In the provided example (Figure 1), student 4 is the only one with a known rank. The students 1, 3, 5 are shorter than student 4 and students 2, 6 are taller than student 4. Any student who cannot establish a strict taller/shorter relationship with every other peer has an unknown rank.

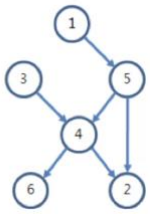

Figure 1

Implement the function `countKnownRankStudents` in `functions.cpp` to return the number of students whose rank can be precisely determined.

3.1. Inputs

- `int n`: The number of students.
- `std::vector<std::pair<int, int>>& comparisons`: A vector with height comparisons. Each pair `{a, b}` indicates that student `a` is shorter than student `b`.

3.2. Output

- The number of students whose relative height order can be determined.

Constraints

```
2 ≤ n ≤ 500
```

Example

```
int n = 6;
std::vector<std::pair<int, int>> comparisons = {{1, 5}, {3, 4},
{5, 4}, {4, 2}, {4, 6}, {5, 2}};
```

Submission Instructions

- For each problem, write your functions in the designated `functions.cpp` file according to the instructions. You may modify other files for testing purposes, but keep in mind that **only** `functions.cpp` **will be graded**, using the original header file `functions.hpp` and `main.cpp`.
- Grading will be conducted using C++20 standard. The `main` function in the autograder, which includes test cases, may differ from the `main` function released to students. The compilation process is as follows:

```
For all problems,
        $ g++ -c main.cpp -o main.o -std=c++20 (using the
      original main.cpp)
        $ g++ -c functions.cpp -o functions.o -std=c++20
```

- When submitting your codes, ensure that all test codes printing out `stdout` are removed, unless explicitly instructed otherwise. **Grading relies solely on the `stdout`, and including any unrelated codes may result in inaccurate grading.**
- You must **NOT** share your code with other students. Additionally, students must not rely on LLMs to generate code directly. Any student found to have a high level of code similarity, as determined by our similarity detection process, may face serious penalties.
- If you submit late, grace days will be automatically deducted. You have 5 grace days available for homework submissions throughout the semester. Grace days are counted in 24-hour increments from the original due time. For example, if an assignment is due on 1/1 at 11:59 PM, submitting it 30 minutes late will use one grace day, while submitting it on 1/3 at 9:00 PM will use two grace days. Late submissions will **NOT** be accepted once all grace days have been used.
- For questions about this homework, use the "Homework 4" discussion forum on eTL.
- Failing to adhere to the submission guidelines may result in penalties applied without exception.